

読み方教科書（自動生成） — index.html

- 以下更新が必要な場合は、ターミナルで実行

```
python3 generate_codemap.py index.html コード地図_自動生成.md GanttForge_お手本実フ
ァイル対応表_2.xlsx
python3 generate_textbook.py index.html 読み方教科書_自動生成.md
```

このファイルは `generate_textbook.py` が生成します。手で編集しないでください。学び方の型（読む順・予測→確認・自己確認）は生成器に固定。各関数・各セクションの「なぜ」は、コード内の【設計判断】コメントから自動抽出しています。コードのコメントを更新して再生成すれば、この教科書も追従します。

再生成：`python3 generate_textbook.py <入力html> <出力md>`（同フォルダに `generate_codemap.py` が必要）

- 生成日時：2026-07-17 15:08
- 総行数：8,048 / 関数：267 / 設計判断ブロック：34

読む順（下から上へ）

下の層は上の層を知らなくても読める。上の層は下の層を知らないと読めない。だから下から。

- 層1：DB基盤（土台） ... 関数4 / 設計判断0
- 層2：単一ストアCRUD ... 関数36 / 設計判断0
- 層3：複合操作（★心臓部） ... 関数7 / 設計判断3
- 層4：UIの背骨 ... 関数25 / 設計判断2
- 層5：パネル群 ... 関数54 / 設計判断5
- 層6：モーダル・ガント操作・マインドマップ ... 関数90 / 設計判断11
- 層7：入出力 JSON / PDF / Excel（最後でよい） ... 関数35 / 設計判断4

層1：DB基盤（土台）

すべての保存処理が乗る土台。ここを固めないと上は読めない。

読む場所

- L1811～ `IndexedDB` 定数（関数0）
- L1821～ `ストア定義`（関数2）
- L1936～ `DB接続の共通処理`（関数2）

読む前に予測する

- なぜ主キーを全ストア共通で "id" に固定し、自動採番を使わないのか。使うと何が困る？
- DBへの接続は毎回開き直すのか、1つを使い回すのか。正しいのはどちらで、なぜ？

このコードが語る「なぜ」（設計判断コメントから自動抽出）

- (この層に設計判断コメントは検出されず)

収録関数

- `createStore` (L1882) — db 上に `storeDefinition.name` のストアと、指定インデックスが作成されていることを保証する。
- `openDB` (L1895) — 成功時：開いた `IDBDatabase` を `resolve` する。
- `getDB` (L1945) — 開いている `IDBDatabase` を1つ返すことを保証する (毎回同じインスタンス)。
- `promisifyRequest` (L1964) — 成功時：`request.result` を `resolve`。失敗時：発生したエラーを `reject`。

自己確認 (言えたら合格)

- 「このアプリのデータはどこに・どんな単位で保存されますか？」に、ストアと主キーの話で答えられる。

層2：単一ストアCRUD

9つのストアが同じ骨格(add / get / delete)を持つ。1つ読めば残りは差分だけで読める。

読む場所

- L1978～ `projects` スタアのCRUD (関数3)
- L2020～ `schedules` スタアのCRUD (関数5)
- L2095～ `tasks` スタア (カンバン) のCRUD (関数4)
- L2146～ `comments` スタアのCRUD (関数4)
- L2203～ `changelog` スタア (変更履歴) のCRUD (関数3)
- L2268～ `issues` スタア (マインドマップ) のCRUD (関数4)
- L2321～ `memos` スタアのCRUD (関数4)
- L2372～ `quickmemos` スタア (即時メモ) のCRUD (関数5)
- L2451～ `snapshots` スタアのCRUD (関数4)

読む前に予測する

- `getXByProject` は全件を舐めるのか、インデックス(`by_projectId`)を使うのか。どう効く？
- `add`系が「新規追加」と「更新」を兼ねられるのはなぜ？ (`put`の性質)

このコードが語る「なぜ」(設計判断コメントから自動抽出)

- (この層に設計判断コメントは検出されず)

収録関数

- `addProject` (L1990) — コミット完了後に `resolve` する (=保存が確定したことを保証)。
- `getAllProjects` (L2002) — プロジェクトの配列を `resolve` する (0件なら空配列)。
- `deleteProject` (L2013) — コミット完了後に `resolve` する (対象が存在しなくても `delete` はエラーにならない=冪等)。
- `addSchedule` (L2033) — コミット完了後に `resolve` する (=保存が確定したことを保証)。
- `getSchedulesByProject` (L2046) — スケジュールの配列を `resolve` する (0件なら空配列)。ルート行・子行の区別はせず全行を返す。

- `getChildSchedules` (L2061) — 子スケジュールの配列を resolve する (子がなければ空配列)。
- `deleteSchedule` (L2076) — コミット完了後に resolve する (対象が存在しなくても delete はエラーにならない=冪等)。
- `getAllSchedules` (L2088) — スケジュールの配列を resolve する (0件なら空配列)。
- `addTask` (L2108) — コミット完了後に resolve する (=保存が確定したことを保証)。
- `getTasksByProject` (L2119) — タスクの配列を resolve する (0件なら空配列)。カンバン列ごとの振り分けは呼び出し側の責務。
- `deleteTask` (L2129) — コミット完了後に resolve する (対象が存在しなくても delete はエラーにならない=冪等)。
- `getAllTasks` (L2139) — タスクの配列を resolve する (0件なら空配列)。
- `addComment` (L2158) — コミット完了後に resolve する (=保存が確定したことを保証)。
- `getCommentsByTask` (L2172) — コメントの配列を resolve する (0件なら空配列)。
- `deleteComment` (L2185) — コミット完了後に resolve する (対象が存在しなくても delete はエラーにならない=冪等)。
- `getAllComments` (L2196) — コメントの配列を resolve する (0件なら空配列)。
- `addChangelogEntry` (L2216) — コミット完了後に resolve する (=保存が確定したことを保証)。
- `getChangelogByProject` (L2228) — 変更履歴の配列を resolve する (0件なら空配列)。
- `clearChangelogByProject` (L2247) — コミット完了後に resolve する (対象が0件でも正常に完了する)。
- `addIssue` (L2278) — コミット完了後に resolve する (=保存が確定したことを保証)。
- `getIssuesByProject` (L2291) — ノードの配列を resolve する (0件なら空配列)。親子どちらのノードも区別せず全件返す。
- `deleteIssue` (L2304) — コミット完了後に resolve する (対象が存在しなくても delete はエラーにならない=冪等)。
- `getAllIssues` (L2314) — ノードの配列を resolve する (0件なら空配列)。
- `addMemo` (L2333) — コミット完了後に resolve する (=保存が確定したことを保証)。
- `getMemosByProject` (L2344) — メモの配列を resolve する (0件なら空配列)。
- `deleteMemo` (L2355) — コミット完了後に resolve する (対象が存在しなくても delete はエラーにならない=冪等)。
- `getAllMemos` (L2365) — メモの配列を resolve する (0件なら空配列)。
- `addQuickMemo` (L2382) — コミット完了後に resolve する (=保存が確定したことを保証)。
- `getQuickMemosByProject` (L2393) — 即時メモの配列を resolve する (0件なら空配列)。
- `deleteQuickMemo` (L2404) — コミット完了後に resolve する (対象が存在しなくても delete はエラーにならない=冪等)。
- `clearQuickMemosByProject` (L2420) — コミット完了後に resolve する (対象が0件でも正常に完了する)。
- `getAllQuickMemos` (L2444) — 即時メモの配列を resolve する (0件なら空配列)。
- `addSnapshot` (L2463) — コミット完了後に resolve する (=保存が確定したことを保証)。
- `getSnapshotsByProject` (L2474) — スナップショットの配列を resolve する (0件なら空配列)。
- `deleteSnapshot` (L2487) — コミット完了後に resolve する (対象が存在しなくても delete はエラーにならない=冪等)。
- `getAllSnapshots` (L2497) — スナップショットの配列を resolve する (0件なら空配列)。

自己確認 (言えたら合格)

- 逆引き表を見ずに、あるストアを触る関数を骨格(add/get/delete)で挙げられる。

層3：複合操作（★心臓部）

層2のCRUDを組み合わせて「意味のある1操作」を作る層。ここが一番説明したくなる所。

読む場所

- L2504～ **複合操作（複数ストアにまたがる処理）の共通ユーティリティ**（関数7）

読む前に予測する

- 子スケジュールの日付を変えたら、親は？ その親の親は？（再帰の必要性を自分で導く）
- 子を全部消したら親の日付はリセットされる？ されない？
- 親を消したら、子・孫・コメントはどうなる？ 変更履歴は1件？ それとも個別？

このコードが語る「なぜ」（設計判断コメントから自動抽出）

- L2582／**祖先日付の再計算をなぜ独立した関数にするか**
 - 基本設計書4.3節「子の開始日・終了日を変更すると、親の日付は全子の最早開始日～最遅終了日に 自動更新され、上位階層へも再帰的に伝播する」は、連鎖削除に限らず、将来スケジュールの日付編集機能（バーのドラッグ等）でも同じ処理が必要になる（詳細設計書3.2.3節 syncParentDates/syncAncestorD...
- L2618／**スケジュールの連鎖削除**
 - 基本設計書L155-157・L200、シーケンス図集「3. 親スケジュール削除」の仕様どおり、1つの複合操作として次の3段階で実装する。1. 読み切り：getSchedulesByProjectでプロジェクト内の全件を1回だけ読み、アプリ層のツリー走査で「起点＋全子孫」のidを集める（データモデル設計6章：複数...
- L2689／**マインドマップノードの連鎖削除**
 - 詳細設計書3.5.3節の仕様どおり、対象ノード＋子孫ノードを収集して削除する。deleteScheduleCascade と同じ考え方だが、issues には次の3点の違いがある：・ comments のような紐づく別ストアが無い（削除するのはissues自体のみ）・ 日付の概念が無いため、祖先の再計算（syncAn...

収録関数

- **generateId** (L2514) — 十分に一意な文字列idを返す。
- **diffFields** (L2531) — [{field, label, before, after}] の配列を返す（1件も変わっていなければ空配列）。
- **recordChangelogEntry** (L2552) — 記録した場合はtrueを、"edit"で変更が無く記録しなかった場合はfalseを返す。
- **getScheduleById** (L2575) — 見つかった場合はそのオブジェクトを、存在しない場合は undefined を resolve する。
- **syncAncestorDates** (L2597) — 祖先日付の再計算をなぜ独立した関数にするか
- **deleteScheduleCascade** (L2632) — スケジュールの連鎖削除
- **deleteIssueCascade** (L2703) — マインドマップノードの連鎖削除

自己確認（言えたら合格）

- 連鎖削除の3段階（全件読み切り→削除＋個別ログ→起点の親の日付再計算）を口で説明できる。

層4：UIの背骨

「純粋関数で計算 → 薄い関数でDOMに書く → 統括関数がつなぐ」の3分割。この型が分かれると全パネルが同じ読み方で読める。

読む場所

- L3223～ UI：状態・純粋関数（DOM非依存）（関数11）
- L4658～ UI：DOM反映（薄い関数）（関数7）
- L4705～ UI：統括（データ取得→純粋関数→DOM反映のつなぎ）（関数7）

読む前に予測する

- なぜ「計算」と「DOMへの反映」を別の関数に分けるのか。混ぜると何が困る？

このコードが語る「なぜ」（設計判断コメントから自動抽出）

- L3224／このセクションの関数は「データ配列を受け取り、データ/文字列を返す」だけの
 - このセクションの関数は「データ配列を受け取り、データ/文字列を返す」だけの 純粋関数と、実際にDOMへ反映する薄い関数に分ける。前者はdocumentに一切触れないため、ブラウザが無い環境（Node.js等）でも配列を渡すだけで動作を検証できる。【今回のスコープ（UIデザイン仕様書・基本設計書2章に基づくメイン画面...
- L4659／以下は「HTML文字列を対応する要素のinnerHTMLへ入れるだけ」の薄い関数に留める。
 - 以下は「HTML文字列を対応する要素のinnerHTMLへ入れるだけ」の薄い関数に留める。ロジック（何を表示するか）は上の純粋関数側に寄せ、ここでは判断を持たせない。

収録関数

- `escapeHtmlText` (L3287) — そのままHTML文字列に埋め込んでよいエスケープ済み文字列を返す。
- `buildScheduleTreeRows` (L3311) — ガントチャートの行として上から順に描画できる配列
- `appendScheduleAndDescendants` (L3332) — —
- `calculateWbsNumbers` (L3357) — スケジュールid→WBS番号文字列 のMapを返す。
- `filterVisibleScheduleTreeRows` (L3374) — 実際に画面へ表示すべき行だけの配列を返す（WBS番号は事前に計算済みのものを使う
- `areAllParentRowsCollapsed` (L3397) — 親行がすべて折りたたまれていればtrue、そうでなければfalseを返す。
- `groupTasksByStatus` (L3405) — カンバンの3列分のタスク配列 {backlog, doing, done} を返す。
- `parseDateStringToUtcTimestamp` (L3417) — その日付のUTC深夜0時のミリ秒タイムスタンプを返す。
- `buildGanttTimelineDays` (L3429) — タイムライン列として左から順に描画できる配列を返す。
- `calculateDayOffsetFromTimelineStart` (L3452) — 経過日数（整数）。範囲外の日付なら負の値、またはタイムラインを超える大きな値になる。
- `formatUtcTimestampToDateString` (L3461) — UTC基準の日付文字列を返す。
- `applyProjectSelectOptions` (L4662) — —
- `applyWbsPanel` (L4666) — —
- `applyGanttPanel` (L4670) — —
- `applyKanbanPanel` (L4674) — —
- `applyMindmapPanel` (L4678) — —
- `applyLockState` (L4688) — 画面上のロック関連の見え目が状態に一致する。
- `applyReportButtonAvailability` (L4700) — 選択されていればdisabledを外し、されていなければdisabledにする。

- `refreshProjectSelectDropdown` (L4710) —  が操作可能になる。
- `handleAddIssueButtonClick` (L7289) — `createMindmapChildNodeWithInlineEdit`と同じ。
- `handleMindmapNodePointerDown` (L7321) — ドラッグ状態が開始される（実際の座標更新は `pointermove` ハンドラが行う）。
- `handleMindmapNodePointerMove` (L7335) — ノードが視覚的にポインタへ追従する。
- `handleMindmapNodePointerUp` (L7350) — ノードの新しい座標がDBに保存され、パネルが再描画される。
- `openMindmapContextMenu` (L7384) — 右クリックメニューが画面上に表示される。
- `closeMindmapContextMenu` (L7395) — 右クリックメニューが閉じる。
- `handleRenameIssueButtonClick` (L7409) — 保存後、変更履歴を記録し、パネルを再描画する。
- `handleAddChildIssueButtonClick` (L7425) — `createMindmapChildNodeWithInlineEdit`と同じ。
- `handleAddSiblingIssueButtonClick` (L7436) — `createMindmapChildNodeWithInlineEdit`と同じ。
- `handleMindmapContextMenuAction` (L7449) — 選択した操作が実行される。
- `handleDeleteIssueButtonClick` (L7473) — 確認OK、または3件以下なら `deleteIssueCascade` で削除し、自動レイアウトを
- `handleToggleIssueCollapseButtonClick` (L7490) — 対象ノードの子孫が表示/非表示になる。
- `handleLockToggleButtonClick` (L7504) — ロック状態がDBに保存され、画面にも反映される。トースト「ロックしました」／
- `handleTodayButtonClick` (L7520) — ガントチャートが今日の日付付近までスクロールされる。
- `handleWbsPanelVerticalScroll` (L7543) — ガント本体がWBSパネルと同じ縦位置までスクロールされる。
- `handleGanttScrollAreaVerticalScroll` (L7555) — WBSパネルがガント本体と同じ縦位置までスクロールされる。
- `handleGanttBackgroundPointerDown` (L7587) — 背景ドラッグの状態が開始される（実際のスクロールは `pointermove` ハンドラが行う）。
- `handleGanttBackgroundPointerMove` (L7602) — ガント本体（および対応表No.41の同期によりWBSパネルも）がポインタに追従してスクロールする。
- `handleGanttBackgroundPointerUp` (L7614) — 背景ドラッグが終了する。
- `handleGranularityButtonClick` (L7624) — ガントチャートが新しい粒度の見出し・列幅で再描画される。
- `handleToggleColumnClick` (L7637) — WBSパネルに列が追加/削除された状態が反映される。
- `handleSwitchToKanbanButtonClick` (L7648) — タスク管理パネルの開閉が行われる。
- `handleOutOfScopeButtonClick` (L7660) — アラートで機能名とスコープ外である旨を表示する（データは変更しない）。
- `initializeApp` (L7670) — 画面を開いた直後から、先頭プロジェクトのデータが3パネルに表示された状態になる

自己確認（言えたら合格）

- バー操作から変更履歴の記録までを、一続きの流れと言える。

層7：入出力 JSON / PDF / Excel（最後でよい）

他から呼ばれるが他をほとんど呼ばない「葉」。理解の土台には要らないので最後に、必要な時だけ。

読む場所

- L2732～ H1：データ出力（JSONエクスポート）（関数4）
- L2794～ H2：データ取込（JSONインポート+差分適用）（関数12）
- L3110～ H3：PDF出力（exportPDF）（関数4）
- L3545～ H4：Excel出力（exportExcel）（関数15）

読む前に予測する

- インポートはなぜ「差分適用」なのか。全消し→全書き込みにしない理由は？（履歴とIDの保持）

このコードが語る「なぜ」（設計判断コメントから自動抽出）

- L2733／対象は基本設計書9.1に明記された全8ストア（**projects/schedules/tasks/issues/**
 - 対象は基本設計書9.1に明記された全8ストア（projects/schedules/tasks/issues/comments/memos/quickmemos/snapshots）。changelog（変更履歴）はこの一覧に含まれないため 対象外とする。crud-implementation.mdの1関数1処理の方...
- L2795／質問リスト_未確認事項.md Q1～Q3の決定に従う：
 - 質問リスト_未確認事項.md Q1～Q3の決定に従う：
 - ・比較・適用対象はH1と同じ全8ストア（changelogは対象外）。
 - ・変更履歴（changelog）への記録は既存のLOGGED_STORES（projects/schedules/tasks/issues）のみに限定し、comments/memos/qui...
- L3111／**crud-implementation.md対象外（DB操作を伴わない）。基本設計書9.3・詳細設計書**
 - crud-implementation.md対象外（DB操作を伴わない）。基本設計書9.3・詳細設計書 3.10.2に従い、以下の方針で実装する：
 - ・キャプチャ対象は#wbsPanel（左：スケジュール一覧）と#ganttpanelBody（右：ガントチャート）。#ganttpanelBodyは.gantt-scro...
- L3546／**crud-implementation.md対象外（DB操作を伴わない）。基本設計書9.4・詳細設計書**
 - crud-implementation.md対象外（DB操作を伴わない）。基本設計書9.4・詳細設計書 3.10.1に従う。日付列・サマリーの期間はいずれもプロジェクト自体のstartDate/endDateを 根拠にする（質問リスト_未確認事項.md Q4：配下スケジュールの実効範囲は使わない。シート1とシート2で...

収録関数

- **collectAllStoresDataForExport**（L2745） — {projects, schedules, tasks, issues, comments, memos, quickm...
- **buildExportBackupFileName**（L2763） — 組み立てたファイル名の文字列を返す。
- **downloadObjectAsJsonFile**（L2773） — ブラウザのダウンロードが実行される（戻り値なし）。
- **handleExportDataButtonClick**（L2787） — ダウンロード実行後、トースト「データをエクスポートしました」を表示する。
- **getBackupStoreDefinitions**（L2809） — 8ストア分の { key, label, getAll, put, del, nameField, logged, fi...
- **isValidImportedBackupShape**（L2825） — 有効な形をしていればtrue、そうでなければfalseを返す。
- **computeStoreDiff**（L2838） — { add: [record...], mod: [{before, after}...], del: [record....
- **buildImportDiff**（L2869） — [{ key, label, nameField, add, mod, del }, ...]（差分のあるストアのみ）を...
- **renderImportDiffItemToHtml**（L2887） — 要素のHTML文字列を返す。
- **renderImportDiffModalToHtml**（L2905） — モーダルパネルにそのまま入れられるHTML文字列を返す。
- **openImportDiffModal**（L2953） — モーダルが操作可能な状態で表示される。

- `applyOneImportItem` (L2982) — 1件の適用 (put/delete、必要なら変更履歴の記録) が完了した時点で resolve する。
- `handleApplyImportButtonClick` (L3013) — 適用後はモーダルを閉じ、プロジェクト一覧・現在の画面を再読み込み、
- `handleImportFileInputChange` (L3066) — 検証NG時は何も変更せず終了する。OK時は差分比較モーダルを開く。
- `buildReportFileName` (L3096) — 組み立てたファイル名の文字列を返す。
- `formatDateToOutputTimestampString` (L3105) — 組み立てた文字列を返す。
- `arePdfLibrariesLoaded` (L3135) — 両方読み込まれていればtrue、そうでなければfalseを返す。
- `captureElementFullContentToCanvas` (L3145) — キャプチャ結果のCanvas要素を resolve する。
- `composeGanttPdfCanvas` (L3160) — 合成済みのCanvas要素を返す。
- `exportPDF` (L3192) — 成功時はトースト「PDFを保存しました」、ライブラリ未読み込み時は
- `flattenScheduleTreeRowsForExcel` (L3579) — [{no, wbsNumber, schedule, depthLevel, depthLabel, durationD...
- `calculateExcelMonthGroups` (L3601) — [{yearMonthLabel, startDayIndex, dayCount}, ...] (startDayInd...
- `stripHashFromHexColor` (L3620) — "#"無しの6桁16進数文字列を返す。
- `buildGanttSheet` (L3631) — `xlsx-js-style`のワークシートオブジェクトを返す。
- `buildSummarySheet` (L3700) — `xlsx-js-style`のワークシートオブジェクトを返す。
- `exportExcel` (L3723) — 完了後、トースト「Excelファイルを保存しました」を表示する。
- `calculateTodayMarkerPosition` (L3743) — 範囲内ならleftPx (数値)、範囲外 (表示中のプロジェクト期間の外) ならnullを返す。
- `calculateBarGeometry` (L3755) — {leftPx, widthPx, durationDays} を返す。日付が無い場合はnullを返す。
- `shiftDateStringByDays` (L3768) — 計算後の日付文字列を返す。
- `calculateDraggedBarDates` (L3787) — クランプ済みの{startDate, endDate}を返す。
- `calculateMilestoneMarkerPositions` (L3822) — {leftPx, name[]} を返す。
- `collectIssueSubtreeIds` (L3836) — [rootIssueId, ...子孫id...] の配列を返す。
- `filterVisibleIssues` (L3865) — 実際に画面へ表示すべきissueだけの配列を返す。
- `buildMindmapEdgeList` (L3882) — {fromIssue, toIssue[]} を返す。
- `calculateEdgeGeometry` (L3899) — {leftPx, topPx, widthPx, angleDeg} を返す。

自己確認 (言えたら合格)

- インポートは `projects/schedules/tasks/issues` だけ変更履歴に記録する、と理由つきで言える。

未分類セクション (層に割り当てられなかったもの)

下記は既知の層に該当しませんでした。新しい種類のコードが増えた可能性があります。

`generate_textbook.py` の `LAYERS` に振り分け先を追記してください。

- L3467 追加実装仕様書1章：営業日・祝日カレンダー
- L4081 追加実装仕様書8.3節：画面フィット

この教科書が古くならない仕組み

「読む場所・収録関数・なぜ」はコードから毎回導出しているので、機能を足して再生成すれば自動で反映されます。特に「なぜ」は【設計判断】コメントを写しているだけなので、**コメントを丁寧に書き続ける限り、教科書の質はコードの質と同じだけ保たれます。**逆に言えば、設計判断コメントを省くと、その関数は教科書から「なぜ」が消えます。コメント規約が、そのまま教科書の生命線です。